LABORATORY

LAB 08

Temperature Measurement with DS18B20

Digital Temperature Monitoring over 1-Wire Using EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1 Introduction . . . 2
1.1 Lab Overview . . . 2
1.2 Objectives . . . 2
2 Background . . . 2
2.1 DS18B20 Digital Temperature Sensor . . . 2
2.2 1-Wire Communication and the DQ Data Line . . . 3
2.3 From Measurement Command to Temperature Value . . . 3
2.4 Resolution and Conversion Time . . . 4
3 Hardware Setup . . . 4
3.1 Required Hardware . . . 4
3.2 Pin Mapping and Module Connections . . . 4
4 EduFramework API . . . 5
4.1 `DS18B20_begin()` . . . 6
4.2 `DS18B20_readCelsius()` . . . 6
5 Lab Exercise . . . 7
5.1 Requirements . . . 7
5.2 Program . . . 7
5.3 Verification . . . 9
6 Extension . . . 9
6.1 Changing the Resolution . . . 9
6.2 Nonblocking Temperature Measurement . . . 10
6.3 Multiple DS18B20 Sensors on One 1-Wire Bus . . . 10
6.4 Other Advanced Features . . . 10
7 References . . . 11

1 Introduction

1.1 Lab Overview

Temperature measurement is a common function in embedded systems, ranging from environmental monitoring to temperature-dependent equipment control. An application may use a single sensor at one location or several sensors distributed throughout a system. In their study of an air-temperature distribution measurement network, Elyounsi and Kalashnikov evaluated DS18B20 sensors alongside Pt100 sensors and highlighted the importance of air conditions, response time, and measurement arrangement when comparing results [2] .

The DS18B20 is a digital temperature sensor that uses the `1-Wire` communication protocol. Unlike an analog sensing element whose voltage must be measured by a microcontroller ADC, the DS18B20 converts temperature internally and transmits digital data over the `DQ` line. It supports resolutions from `9 bit` to `12 bit` and has a unique `64-bit` identification code for multi-device applications [1] .

This lab uses a **three-pin DS18B20 module** and the MaaZEDU Development Board. EduFramework provides a device-level API, so the application does not need to generate the timing pulses required by the `1-Wire` protocol. The program initializes one sensor, reads temperature in degrees Celsius, and displays the readings periodically on the Serial Monitor. Multiple-device management, resolution configuration, and nonblocking measurement are reserved for the Extension section [5] .

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the characteristics of the DS18B20 and distinguish digital temperature readings from analog signals acquired through an ADC.
2. Explain the roles of the `DQ` data line, pull-up resistor, and ROM identification code in `1-Wire` communication.
3. Distinguish resolution from accuracy and recognize how sensor conversion time depends on the selected resolution.
4. Use `DS18B20_begin()` and `DS18B20_readCelsius()` to acquire temperature measurements with EduFramework.
5. Develop and verify a temperature-monitoring application and identify unsuccessful sensor initialization or read operations.

2 Background

2.1 DS18B20 Digital Temperature Sensor

The DS18B20 integrates temperature sensing and conversion into digital data. The microcontroller requests a measurement and then receives the encoded temperature through a digital interface. According to the datasheet, the sensor has a measurement range of `-55 °C` to `+125 °C`, a specified accuracy of `±0.5 °C` from `-10 °C` to `+85 °C`, and an external supply-voltage range of `3.0 V` to `5.5 V` [1] .

Characteristic	DS18B20 Datasheet Specification
Measured quantity	Temperature in degrees Celsius.
Measurement range	`-55 °C` to `+125 °C`.
Accuracy	`±0.5 °C` from `-10 °C` to `+85 °C`.
Resolution	`9 bit`, `10 bit`, `11 bit`, or `12 bit`.
Interface	`1-Wire`, using one `DQ` data line and a common GND.
Identification	Each sensor has a unique `64-bit` ROM code.

Table 8.2.1. Key characteristics of the DS18B20

In Labs 04 and 06, the application processes ADC values obtained from analog signals. With the DS18B20, the result is not obtained through `analogRead()` but through the sensor's digital commands and data. Consequently, the application can work directly with temperature values instead of converting an input voltage [1] [5].

Note

> ***Resolution*** *describes the smallest representable increment in the temperature data.* ***Accuracy*** *concerns the deviation from the actual temperature or a reference value. Displaying more decimal places does not guarantee a corresponding measurement accuracy [1].*

2.2 1-Wire Communication and the DQ Data Line

`1-Wire` is a serial communication protocol in which the master and devices on a bus share a single data line. The devices also share GND as their voltage reference. Devices pull the data line low and release it; a pull-up resistor restores the bus to a high level when no device is pulling it low. Therefore, a pull-up resistor is an essential part of the `1-Wire` connection [3].

A typical transaction begins with a master reset and a device presence response, followed by device-selection and function commands. Individual bits are transferred in defined timing intervals, or **time slots**. Analog Devices describes four fundamental operations: reset, write `0`, write `1`, and read a bit [3].

Each DS18B20 also contains a unique `64-bit` ROM code. This code allows the master to distinguish sensors connected in parallel on the same bus. Thus, one data line can support multiple measurement locations, provided that the wiring network and communication satisfy the electrical requirements of `1-Wire` [1].

The main exercise uses only one module, so the application does not need to search for and store ROM codes manually. `DS18B20_begin()` discovers the sensor and establishes communication through the EduFramework OneWire driver [5].

2.3 From Measurement Command to Temperature Value

To obtain a new reading, the DS18B20 must perform a temperature conversion. According to the datasheet, the `Convert T` command initiates conversion. When it is complete, the temperature data are stored in the **scratchpad**, a memory area that also contains configuration bytes and a `CRC8` verification byte. The data can then be retrieved using `Read Scratchpad` [1].

EduFramework encapsulates these operations in `DS18B20_readCelsius()`: it starts conversion, waits for completion, reads the scratchpad, verifies `CRC8`, and decodes the data into degrees Celsius. The application receives a `float` value without manipulating individual protocol bits [5].

The data-acquisition sequence in this lab is summarized as follows:

`Temperature` → `DS18B20` → `1-Wire (DQ)` → `DS18B20_readCelsius()` → `Serial1` → `Serial Monitor`.

2.4 Resolution and Conversion Time

The DS18B20 resolution can be selected from `9 bit` to `12 bit`. According to the manufacturer, higher resolution provides a smaller quantization increment but increases the maximum conversion time. The following table presents the datasheet values [1].

Resolution	Temperature Increment	Maximum Conversion Time
`9 bit`	`0.5 °C`	`93.75 ms`
`10 bit`	`0.25 °C`	`187.5 ms`
`11 bit`	`0.125 °C`	`375 ms`
`12 bit`	`0.0625 °C`	`750 ms`

Table 8.2.2. DS18B20 resolution and conversion time

A displayed value such as `28.063 °C` may represent a temperature encoded in `0.0625 °C` increments and rounded for printing. The number of decimal places must not be used to infer sensor accuracy. Furthermore, application-level reading time includes communication and driver processing, so the display interval is not exactly equal to the conversion time shown in the table.

3 Hardware Setup

3.1 Required Hardware

The exercise uses a MaaZEDU Development Board based on the `S32K144` microcontroller and a three-pin DS18B20 module. On the module used here, the pins are labeled `+`, `OUT`, and `-`, corresponding to the positive supply, data line, and GND. The program was verified with a `3.3 V` supply and one sensor connected to `GPIO7` [6] [5].

Hardware	Description
MaaZEDU Development Board	Board based on the S32K144 microcontroller with accessible GPIO pins.
DS18B20 Temperature Sensor Module	Three-pin module containing a DS18B20 and supporting PCB components.
Jumper Wires	Three wires connecting power, GND, and data between the module and MaaZEDU.
USB Cable	Connects the board to a computer for programming and Serial Monitor access.

Table 8.3.1. Hardware required for the exercise

3.2 Pin Mapping and Module Connections

The MaaZEDU Development Board Guide identifies a connector GPIO as `GPIO7`. In an EduFramework application, `GPIO7` is the **Logical Pin** passed to the API to select the `1-Wire` data line.

Component	Logical Pin	MCU Pin	Function
DS18B20 `OUT`	`GPIO7`	`PTD9`	Digital I/O / 1-Wire data; MCU port designation according to MaaZEDU Guide 1.0.

Table 8.3.2. Data pin used in Lab 08

DS18B20 Module	MaaZEDU	Function
`+`	`3.3V`	Provides external power to the sensor.
`OUT`	`GPIO7`	Transmits and receives `1-Wire` data.
`-`	`GND`	Provides a common voltage reference.

Table 8.3.3. Three-pin DS18B20 module connections

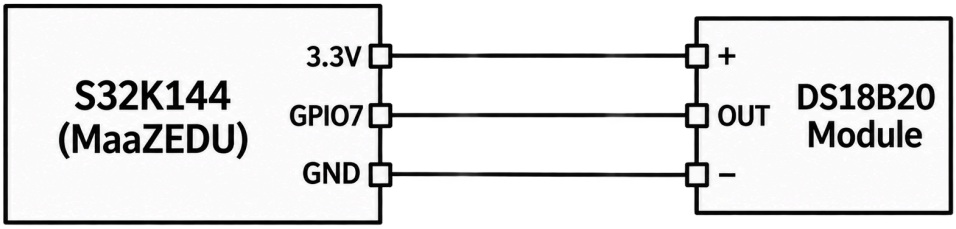


Figure 8.3.1. Wiring diagram of the DS18B20 module and MaaZEDU Development Board

The module used in this exercise has supporting components already mounted on its PCB. A `1-Wire` circuit requires an appropriate pull-up path between the data line and the supply. If a standalone DS18B20 or a different module is substituted, inspect its actual circuit and add an external pull-up resistor if necessary; not all modules should be assumed to have identical resistor configurations [3] .

> **WARNING**
>
> Disconnect power before changing the wiring. Verify the `+`, `OUT`, and `-` markings on the actual module; do not infer the pinout from wire colors or from the pin order of a TO-92 DS18B20 IC. This exercise uses `3.3 V` to match the board's logic level.

4 EduFramework API

EduFramework v3.1.1 provides two groups of DS18B20 functions: a **Simple API** for one sensor and an **Advanced API** for explicit bus and device management. The main exercise uses only `DS18B20_begin()` and

`DS18B20_readCelsius()`. ROM operations, scratchpad access, and resolution configuration are addressed in the Extension section [5].

4.1 `DS18B20_begin()`

`DS18B20_begin()` configures a GPIO line and searches for a DS18B20 on the `1-Wire` bus. When a sensor is found, the driver checks its data and prepares the device for subsequent readings. The API accepts a Logical Pin supporting Digital I/O [5].

`DS18B20_begin`

Initialize `1-Wire` communication and automatically select the first DS18B20 discovered on the specified pin.

Syntax

```
DS18B20_begin(pin);
```

Parameters

Parameter	Type / accepted value	Description
`pin`	Digital Logical Pin	Pin connected to the module's `OUT` terminal, such as `GPIO7`.

Return value

`true` if the sensor is found and initialized successfully; `false` if initialization or device verification fails.

Example:

```
bool ready = DS18B20_begin(GPIO7);
```

In this exercise, the return value is checked immediately after initialization. The program does not attempt to read temperature unless the sensor is ready.

4.2 `DS18B20_readCelsius()`

`DS18B20_readCelsius()` initiates a temperature conversion, waits for the sensor to complete it, and then reads and decodes the result. This is a **blocking** operation: the function does not return while the sensor is still performing the conversion [5].

`DS18B20_readCelsius`

Perform a new temperature measurement and return the result in degrees Celsius.

Syntax

```
DS18B20_readCelsius();
```

Return value

Temperature in degrees Celsius on success; `NAN` on failure. `NAN` is not a valid temperature reading.

Example:

```
float temperature = DS18B20_readCelsius();
```

To distinguish valid readings from errors, the program uses `isnan()` from `math.h`. This function checks for **Not a Number** (`NAN`) before the value is displayed on the Serial Monitor. Detailed error diagnostics through `DS18B20_lastStatus()` are introduced in the Extension section.

The functions `Serial1_begin()`, `Serial1_print()`, `Serial1_println()`, and `Serial1_printFloat()` belong to the EduFramework Serial API and are reused from Lab 03. Lab 08 does not repeat their detailed syntax documentation [5].

5 Lab Exercise

5.1 Requirements

Develop a program that monitors temperature using a DS18B20 and displays readings on the Serial Monitor. The program must meet the following requirements:

- Use a DS18B20 module connected to `GPIO7` and powered at `3.3 V`.
- Initialize `Serial1` at `9600` baud and display a startup message.
- Check sensor initialization before attempting any measurements.
- Read temperature in degrees Celsius and display only valid readings.
- Wait `1000 ms` after each read, then repeat.

Once the system is operating steadily, gently change the temperature at the sensor by touching it or bringing it near a safe heat source. Observe the temperature increase and its subsequent decrease after the heat source is removed. No fixed temperature value is required, since ambient conditions and sensor placement may differ [2].

5.2 Program

Implement the following hardware-verified program in `src/main.c`:

```c
#include "Arduino.h"
#include "ds18b20.h"

#include <math.h>

int main(void)
{
    float temperature = 0.0F;

    setup();

    Serial1_begin(9600U);
    Serial1_println("DS18B20 Temperature Monitor");

    if (false == DS18B20_begin(GPIO7))
    {
        Serial1_println("DS18B20 initialization failed!");

        while (1)
        {
            delay(1000U);
        }
    }

    Serial1_println("DS18B20 initialized successfully.");

    while (1)
    {
        temperature = DS18B20_readCelsius();

        if (false == isnan(temperature))
        {
            Serial1_print("Temperature: ");
            Serial1_printFloat(temperature);
            Serial1_println(" C");
        }
        else
        {
            Serial1_println("Temperature reading failed!");
        }

        delay(1000U);
    }

    return 0;
}
```

Listing 8.5.1. Temperature-monitoring program using the DS18B20 module

`setup()` initializes the underlying EduFramework components; clock, timer, and UART mechanisms for the S32K1xx family are described by NXP in AN5413 [4] . `Serial1_begin(9600U)` configures the Serial Monitor channel, while `DS18B20_begin(GPIO7)` initializes the bus and selects the sensor. If initialization fails, the program displays an error message and does not enter the measurement loop.

Inside the main loop, `DS18B20_readCelsius()` performs a new measurement. The result is checked using `isnan()`. A valid reading is displayed through `Serial1_printFloat()`, whereas an error is reported separately. `delay(1000U)` then inserts a waiting interval before the next measurement.

Note

*`delay(1000U)` specifies a delay **after** each reading; it does not establish an exact `1000 ms` measurement period. At `12-bit` resolution, a DS18B20 conversion can take up to `750 ms` according to the datasheet. Communication and processing introduce additional time between displayed readings [1] [5] .*

5.3 Verification

Build and upload the program to the MaaZEDU Development Board, then open the Serial Monitor at `9600` baud. Check the following:

- The startup message and successful initialization message appear.
- Temperature readings update repeatedly; `NAN` is not printed as a measurement.
- Under stable thermal conditions, successive readings remain close.
- Warming the sensor causes an increasing trend; cooling it causes a decreasing trend.

The following output was recorded during the hardware test for this lab:

```
DS18B20 Temperature Monitor
DS18B20 initialized successfully.
Temperature: 28.063 C
Temperature: 28.063 C
Temperature: 28.000 C
Temperature: 28.000 C
Temperature: 27.938 C
Temperature: 27.813 C
```

These values are observations from one test, not required reference temperatures. As discussed by Elyounsi and Kalashnikov, heat-transfer conditions and sensor response time can affect comparisons between nearby sensors. Therefore, this exercise must not be treated as a calibration procedure [2] .

EXPECTED RESULT

The DS18B20 module initializes successfully, and the Serial Monitor displays temperature readings in degrees Celsius over successive measurements. When the temperature at the sensor changes, the displayed values follow the corresponding trend. The program does not treat failed readings as valid temperatures.

6 Extension

The Simple API is suitable for applications with one sensor and a sequential measurement process. The DS18B20 also supports additional protocol features exposed through the EduFramework Advanced API. The following topics are optional investigations, not requirements of the main exercise [1] [5] .

6.1 Changing the Resolution

According to the datasheet, the DS18B20 supports resolutions from `9 bit` to `12 bit`. Reducing the resolution decreases the maximum conversion time but increases the temperature increment. In EduFramework, `DS18B20_SetResolution()` is part of the Advanced API and takes a selected device and the desired resolution. The predefined values are `DS18B20_RESOLUTION_9_BIT`, `DS18B20_RESOLUTION_10_BIT`, `DS18B20_RESOLUTION_11_BIT`, and `DS18B20_RESOLUTION_12_BIT` [5] .

Extension task: Compare the update time and temperature increments at `9 bit` and `12 bit`. Using the configuration API requires the Advanced API to create a bus context, discover the ROM code, and select the device. A Logical Pin cannot be passed directly to `DS18B20_SetResolution()`.

6.2 Nonblocking Temperature Measurement

In the main program, `DS18B20_readCelsius()` waits for conversion to finish. An application that must also process buttons, update a display, or execute time-sensitive tasks may need to separate conversion initiation from result retrieval. The Advanced API provides this sequence:
`DS18B20_StartConversion()` → `DS18B20_ConversionReady()` → `DS18B20_ReadLastCelsius()`.
The first function starts conversion, the second checks whether the result is ready, and the third reads the converted value. The result must only be read after conversion is complete. This approach can be combined with `millis()` to organize nonblocking application logic, although the Advanced API requires managing `DS18B20_Bus_t` and `DS18B20_Device_t` [5].
Extension task: Design an application that continues to change an LED state while the sensor is converting temperature, without using `delay()` to wait for completion.

6.3 Multiple DS18B20 Sensors on One 1-Wire Bus

Every DS18B20 has a unique `64-bit` ROM code, allowing the master to identify multiple sensors connected to the same data line. This property is also relevant to research on distributed temperature measurement networks [1] [2].
In EduFramework, `DS18B20_FindDevices()` enumerates ROM codes, while `DS18B20_Attach()` associates a discovered sensor with a device context. Unlike the Simple API, this approach enables an application to select a specific sensor when several devices share the bus [5].
Extension task: If at least two DS18B20 sensors are available, inspect each ROM code and develop a program that displays temperature readings by sensor ID. Verify the power supply, pull-up path, and physical wiring limitations before extending the bus.

6.4 Other Advanced Features

EduFramework v3.1.1 also provides functions for device diagnostics and configuration:

- `DS18B20_lastStatus()` and `DS18B20_StatusString()` help identify and display error conditions, such as a missing device or an invalid CRC.
- `DS18B20_ReadScratchpad()` provides access to the sensor's data bytes; the driver verifies CRC8 before accepting the result.
- `DS18B20_SetAlarmThresholds()` and `DS18B20_FindAlarms()` support alarm threshold configuration and discovery of devices in an alarm state.
- `DS18B20_CopyScratchpad()` and `DS18B20_RecallEEPROM()` save and restore sensor configuration in EEPROM.

These functions correspond to the ROM, scratchpad, CRC, and alarm capabilities described in the datasheet [1] [5].

Note

The current EduFramework OneWire driver requires an external supply through `VDD` for temperature conversion or EEPROM programming. Parasite-power operation requiring a strong pull-up is not yet supported. Do not repeatedly write EEPROM inside the main loop because its write endurance is limited [1] [5].

7 References

[1] Maxim Integrated / Analog Devices, *DS18B20 - Programmable Resolution 1-Wire Digital Thermometer* , DS18B20, Rev. 6, 2019.

[2] Ali Elyounsi and Alexander N. Kalashnikov, *Evaluating Suitability of a DS18B20 Temperature Sensor for Use in an Accurate Air Temperature Distribution Measurement Network* , Engineering Proceedings, 10(1), 56, 2021.

[3] Maxim Integrated / Analog Devices, *1-Wire Communication Through Software* , Analog Devices Technical Article, 2002.

[4] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.

[5] EduFramework, *DS18B20 Device API and OneWire Driver* , EduFramework Source Code, v3.1.1.

[6] FPT Software, *MaaZEDU Development Board Guide* , Rev. 1.0, 2025.